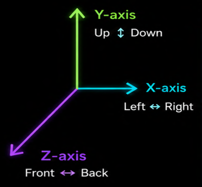
**Hitesh Rathore (HR) Notes**

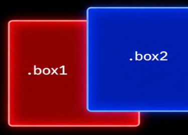
LINKEDIN LINK:
https://www.linkedin.com/in/hitesh-rathore-835279305/

# z-index

Controls the **STACKING ORDER** of elements on the Z-axis (depth axis)


Y-axis
Up ↕ Down
X-axis
Left ↔ Right
Z-axis
Front ↔ Back

## BASIC EXAMPLE



```
.box1 {
  position: relative;
  z-index: 1;
}
.box2 {
  position: relative;
  left: 100px;
  top: -100px;
  z-index: 2;
}
```

→ Since z-index: 2 > z-index: 1

Blue box (.box2) appears above the red box (.box1)

## QUICK RULES

- ✓ Works on positioned elements relative, absolute, fixed, sticky
- ✓ Higher z-index appears on top
- ✓ Can use positive, zero, and negative values
- ✓ Same z-index? HTML order decides
- ✓ Children cannot escape their parent's stacking context

## RULE #1

### WORKS ONLY ON POSITIONED ELEMENTS

✗ Won't work

```
.box {
 z-index: 100;
}
```

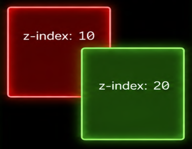
(No positioning)

✓ Works

```
.box {
 position: relative;
 z-index: 100;
}
```

## RULE #2

### HIGHER z-index COMES ON TOP

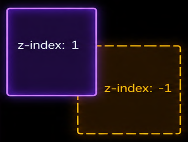

z-index: 10
z-index: 20

Since 20 > 10

Green box appears on top

## RULE #3

### NEGATIVE z-index

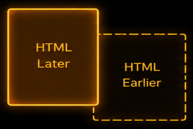

z-index: 1
z-index: -1

Since -1 < 1

Yellow box goes behind

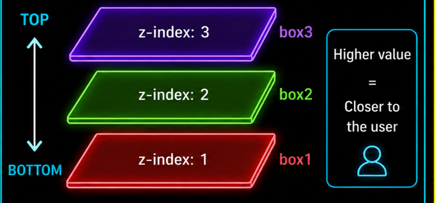
## RULE #4

### SAME z-index

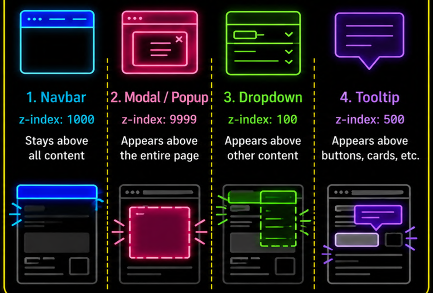

HTML
Later
HTML
Earlier

Same z-index?
HTML order decides.
Later element appears on top

## VISUAL STACKING ORDER


TOP
z-index: 3
box3
z-index: 2
box2
z-index: 1
box1
BOTTOM
Higher value
=
Closer to
the user

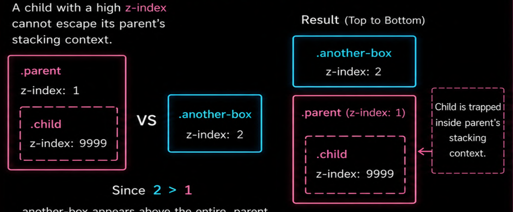
## REAL-LIFE USES



| 1. Navbar | 2. Modal / Popup | 3. Dropdown | 4. Tooltip |
|---|---|---|---|
| z-index: 1000 | z-index: 9999 | z-index: 100 | z-index: 500 |
| Stays above all content | Appears above the entire page | Appears above other content | Appears above buttons, cards, etc. |

## STACKING CONTEXT (IMPORTANT!)

A child with a high z-index cannot escape its parent's stacking context.


.parent
z-index: 1
.child
z-index: 9999
VS
.another-box
z-index: 2
Result (Top to Bottom)
.another-box
z-index: 2
.parent (z-index: 1)
.child
z-index: 9999
Child is trapped
inside parent's
stacking
context.

Since 2 > 1

.another-box appears above the entire .parent (and its child)

## COMMON INTERVIEW QUESTIONS

**Q.** Which element appears on top?

```
.box1 {
  position: relative;
  z-index: 100;
}
```

VS

```
.box2 {
  position: relative;
  z-index: 200;
}
```

**Answer:** .box2 (because 200 > 100)

**Q.** Which element appears on top?

```
.box1 {
  position: relative;
  z-index: -1;
}
```

VS

```
.box2 {
  position: relative;
  z-index: 0;
}
```

**Answer:** .box2 (because 0 > -1)

## COMMON MISTAKES

1. No Position Property

```
.box {
 z-index: 100;
}
```

✗ Doesn't work

2. Using Extremely Large Values

```
z-index:
999999999;
```

✗ Avoid this

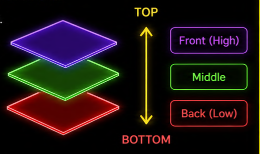
3. Ignoring Stacking Context

Parent (z-index: 1)
Child (z-index: 9999)

⊗ Child can't escape parent context

## SUMMARY

- ✓ z-index controls depth (front/back).
- ✓ Higher z-index → appears on top
- ✓ Lower z-index → goes behind
- ✓ Works on: relative, absolute, fixed, sticky
- ✓ Accepts: positive values, zero, negative values
- ✓ Same z-index → HTML order decides
- ✓ Stacking Context → children cannot escape parent's layer


TOP
Front (High)
Middle
Back (Low)
BOTTOM

z-index controls the Z-axis (Front ↔ Back)

**Remember:** X-axis = Left ↔ Right | Y-axis = Up ↕ Down | Z-axis = Front ↔ Back → z-index controls the Z-axis.

---

# CSS OVERFLOW

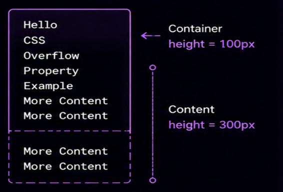
The overflow property controls what happens when content becomes larger than its container.

If content exceeds the box size, CSS must decide:

Show everything? Hide extra content? Add scrollbars?

This is exactly what overflow controls.

## VISUAL IDEA


Hello
CSS
Overflow
Property
Example
More Content
More Content
More Content
More Content
Container
height = 100px
Content
height = 300px

Now content is overflowing.

## SYNTAX

```
overflow: visible;
overflow: hidden;
overflow: scroll;
overflow: auto;
```

| | |
|---|---|
| `visible` | Default. Content is visible outside the box. |
| `hidden` | Extra content is clipped and not visible. |
| `scroll` | Scrollbars are always visible. |
| `auto` | Scrollbars appear only when needed. |

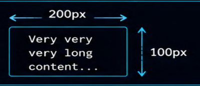
## EXAMPLE BOX

HTML

```
<div class="box">
  Very very very long content...
</div>
```

CSS

```
.box{
   width:200px;
   height:100px;
}
```


200px
Very very
very long
content...
100px

## 1 overflow: visible (Default)

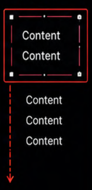
Default value.

```
.box{
  overflow: visible;
}
```

**EXAMPLE**

```
<div class="box">
  Very Long Content...
</div>
.box{
  width:200px;
  height:100px;
  border:2px solid red;
  overflow: visible;
}
```

**RESULT**


Content
Content
Content
Content
Content

- ✓ Content comes outside container.
- ✓ Nothing gets hidden.

## 2 overflow: hidden

Extra content disappears.

```
.box{
  overflow: hidden;
}
```

**EXAMPLE**

```
.box{
  width:200px;
  height:100px;
  overflow:hidden;
}
```

**RESULT**

Content
Content
Content

(hidden below)

Everything outside box becomes invisible.

**USE CASES**

**A) Hide Extra Text**

```
.card{
  overflow:hidden;
}
```

**B) Rounded Images** With

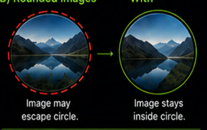


Image may escape circle.
Image stays inside circle.

```
.profile{
  width:200px;
  height:200px;
  border-radius:50%;
  overflow:hidden;
}
```

## 3 overflow: scroll

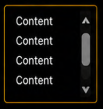
Always shows scrollbars.

```
.box{
  overflow: scroll;
}
```

**EXAMPLE**

```
.box{
  width:200px;
  height:100px;
  overflow:scroll;
}
```

**RESULT**


Content
Content
Content
Content

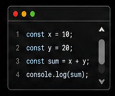
Scrollbar appears even if content fits.

**USE CASES**

Used when you always want scrolling behavior.

**EXAMPLE:**

```
.code-editor{
  overflow:scroll;
}
```



```
const x = 10;
const y = 20;
const sum = x + y;
console.log(sum);
```

**PROBLEM**

Even if content is small:

`overflow:scroll;`

Scrollbar still appears.
Not ideal most of the time. ☹

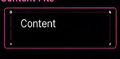
## 4 overflow: auto

Most commonly used.

```
.box{
  overflow:auto;
}
```

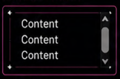
**BEHAVIOR**

**Content Fits**



No scrollbar.

**Content Overflows**


Content
Content
Content

Scrollbar appears automatically.

**USE CASES**

**Chat Applications**

```
.chat{
  height:500px;
  overflow:auto;
}
```

**Dashboard Panels**

```
.sidebar{
  overflow:auto;
}
```

**Code Blocks**

```
pre{
  overflow:auto;
}
```

Very common.

**EXAMPLE:**

```
<pre>
  Very long code line...
</pre>
```

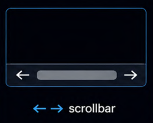
Horizontal scrollbar appears if needed.

## 5 overflow-x and overflow-y

Control directions separately.

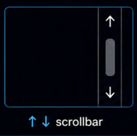
**Horizontal Overflow**

`overflow-x: auto;`



**Vertical Overflow**

`overflow-y: auto;`



**Example**

```
.box{
  overflow-x: hidden;
  overflow-y: auto;
}
```

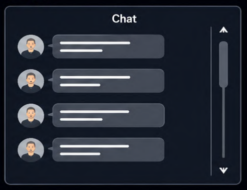
**Meaning:**

- ✓ Horizontal overflow hidden
- ✓ Vertical scrolling allowed

## 6 Real-Life Example: Chat App

```
.chat{
  height: 500px;
  overflow-y: auto;
  overflow-x: hidden;
}
```

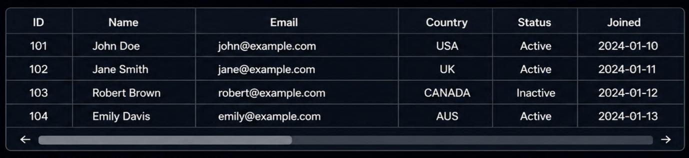
Users can scroll messages vertically.
Cannot scroll sideways.



## 7 Real-Life Example: Table

```
<div class="table-wrapper">
  <table> ... </table>
</div>
```

```
.table-wrapper{
  overflow-x: auto;
}
```



| ID | Name | Email | Country | Status | Joined |
|---|---|---|---|---|---|
| 101 | John Doe | john@example.com | USA | Active | 2024-01-10 |
| 102 | Jane Smith | jane@example.com | UK | Active | 2024-01-11 |
| 103 | Robert Brown | robert@example.com | CANADA | Inactive | 2024-01-12 |
| 104 | Emily Davis | emily@example.com | AUS | Active | 2024-01-13 |

When table becomes wider than screen: ←——→

Horizontal scrolling appears.
Very common in responsive websites.

## 8 Important Interview Concept

Why Doesn't overflow Work?

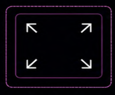
**EXAMPLE:**

```
.box{
  overflow:auto;
}
```

**BUT:**

```
.box{
  height:auto;
}
```

=



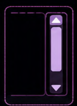
No scrolling appears.

**WHY?**

Because container keeps growing.
Nothing overflows.

**TO SEE SCROLLING:**

```
.box{
  height:300px;
  overflow:auto;
}
```



Now content can exceed height.

## 9 Common Modern Uses

**Cards**

```
.card{
  overflow:hidden;
}
```

**Chat Windows**

```
.chat{
  overflow-y:auto;
}
```

**Tables**

```
.table{
  overflow-x:auto;
}
```

**Code Blocks**

```
pre{
  overflow:auto;
}
```

**Image Galleries**

```
.gallery{
  overflow-x:auto;
}
```

**Sliders**

```
.slider{
  overflow:hidden;
}
```

Only one slide visible at a time.

**Dashboards**

```
.dashboard{
  overflow:auto;
}
```

**Scrollable Sections**

```
.section{
  overflow:auto;
}
```

## 10 Question

**Q1**

`overflow:hidden;`

What happens?

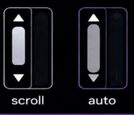
**Answer:**
Extra content becomes invisible.

**Q2** Difference between:

`overflow:scroll;` and & `overflow:auto;`

**Answer:**

- scroll → scrollbar always visible
- auto → scrollbar only when needed


scroll
auto

**Q3** Which is most commonly used?

`overflow:auto;`

**Q4** Can overflow work without width/height restrictions?

**Answer:**
Generally no practical overflow occurs, because the container expands to fit content.

## 11 Quick Summary

| Value | Behavior |
|---|---|
| `visible` | Content escapes container |
| `hidden` | Content clipped |
| `scroll` | Always show scrollbars |
| `auto` | Show scrollbars only when needed |

**Direction Control**

| | |
|---|---|
| `overflow-x` | Controls horizontal overflow |
| `overflow-y` | Controls vertical overflow |

☆ Overflow is all about managing extra content smartly inside containers! 🚀

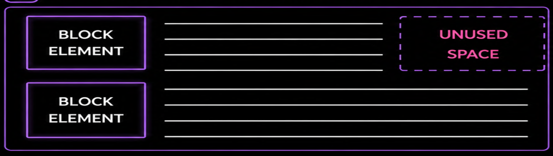
## Concept: Next Element Does NOT Wait for Overflowed Content

If the first element overflows, the next element will NOT start after the overflowed content.
It will start from its original position (just below the first element's box).

# FLOAT

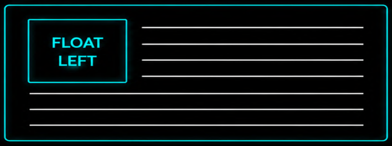
Float is a CSS property that moves an element to the left or right side of its container so that surrounding content can wrap around it and fill the available space.

### 1 NORMAL BLOCK-LEVEL BEHAVIOR (DEFAULT)


BLOCK ELEMENT
UNUSED SPACE
BLOCK ELEMENT

- Block-level elements normally start on a new line.
- Even when free space exists beside a block-level element, the next block-level element moves to the next line.
- As a result, the free space remains unused.

## 2 USING FLOAT



`float: left;`

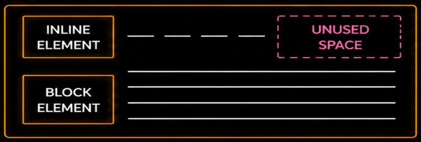
- The element moves to the left side.
- Content flows around it and uses the available space on the right.

**FLOAT VALUES**

`float: left;`
+
`float: right;`

Moves element to the left or right side.

## 3 INLINE + BLOCK ELEMENT SCENARIO

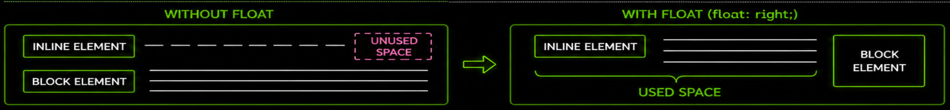

INLINE ELEMENT
UNUSED SPACE
BLOCK ELEMENT

- An inline element does not occupy the entire width.
- However, a block-level element that comes after it still starts on a new line.
- Therefore, the available horizontal space remains unused.

**APPLYING FLOAT**

`float: right;`
↓

- The element moves to the right side.
- Content flows on the left side and uses the remaining available space.


WITHOUT FLOAT
INLINE ELEMENT
UNUSED SPACE
BLOCK ELEMENT
WITH FLOAT (float: right;)
INLINE ELEMENT
BLOCK ELEMENT
USED SPACE

## 4 REAL-WORLD EXAMPLE

**WITHOUT FLOAT**

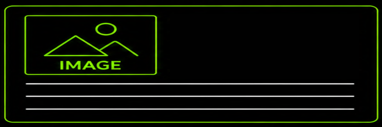


Image is on top and text starts below it.
The space beside the image is not used.

**WITH FLOAT (float: left;)**

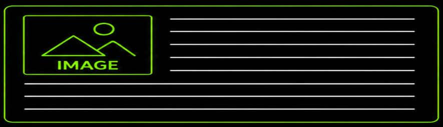


Image floats to the left and text wraps around it, using the available space efficiently.

### KEY TAKEAWAY

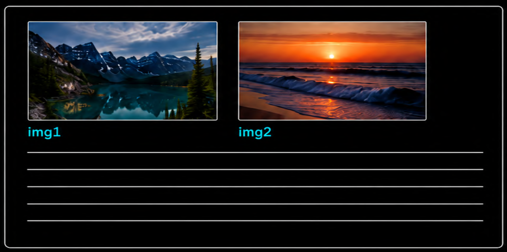
- ✓ **Without Float:** Block-level elements stack vertically and leave unused horizontal space.
- ✓ **With Float:** Elements move to the left or right, allowing surrounding content to wrap around them and fill the available space.

### IMPORTANT NOTES

- Floated elements are removed from the normal document flow.
- Use the clear property to control how elements behave around floats.
- Float is mainly used for wrapping content around images or creating simple layouts.

# FLOAT – Concept Question

### INITIAL LAYOUT (Starting Point)



**? Scenario**

A newspaper article with two images on top and text below.
Let's see how float affects the layout.

**Note**

All text lines represent normal content (paragraphs) in the document.

### 1 QUESTION 1

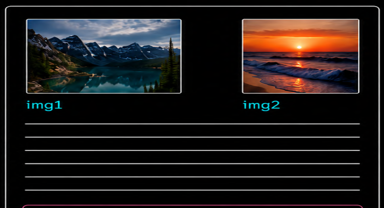
**CSS**

```
.img2 {
  float: right;
}
```

**Predict:**

- What will happen?
- Will only img2 move?
- Will img1 remain at the left?
- Where will the text appear?

**OUTPUT – QUESTION 1**



☆ Text will not come between the images. It will remain below the images, because img1 has not yet given permission to that text.

**REASON – QUESTION 1**

```
.img2 {
  float: right;
}
```

- .img2 is floated to the right as far as possible.
- .img1 is not floated, so it stays in its normal position on the left.

**RESULT**

img1 → Left (normal)
img2 → Right (floated)

### 2 QUESTION 2

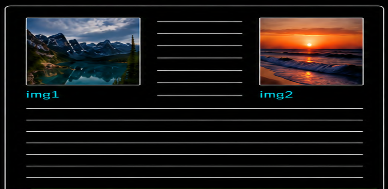
**CSS**

```
.img1 {
  float: left;
}
.img2 {
  float: right;
}
```

**Predict:**

- What will happen now?
- Does img1 move?
- Does img2 move?
- Does the layout change?

**OUTPUT – QUESTION 2**



☆ Text WILL come between the images now. This is because both images have been given permission using float. So the text can flow in the space between them.

**REASON – QUESTION 2**

```
.img1 {
  float: left;
}
.img2 {
  float: right;
}
```

- .img1 is floated left, but it was already on the left, so no visual change.
- .img2 is floated right.

**RESULT**

img1 → Left (floated)
img2 → Right (floated)

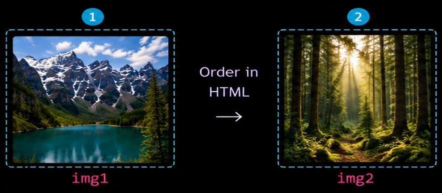
**IMPORTANT NOTE**

When both images are floated (left and right), the text can float in the space between them because it has permission from both sides.

# FLOAT RIGHT – IMPORTANT CONCEPT

**HTML**

```
<body>
   <img class="img1" src="mountain.jpg" alt="img1">
   <img class="img2" src="forest.jpg" alt="img2">
</body>
```


1
Order in
HTML
→
2
img1
img2

**CSS**

```
.img1 {
    float: right;
}

.img2 {
    float: right;
}
```

☆ Both images are given `float: right;`

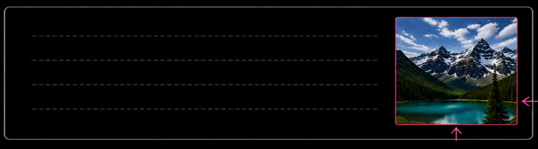
Let's see what happens and why.

## HOW BROWSER WORKS (Execution Flow)

**STEP 1**

Browser encounters first image (img1 - Mountain).
It has `float: right;` so it goes to the extreme right side.

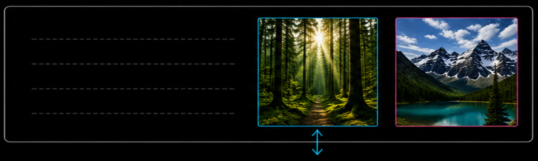
↓



Mountain takes the extreme right position.

**STEP 2**

Browser encounters second image (img2 - Forest).
It also has `float: right;`.
But the extreme right is already occupied by Mountain.
So Forest goes to the left side of Mountain.



Forest gets the next available space.

### FINAL OUTPUT IN BROWSER

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

img1

img2

### KEY OBSERVATION

When multiple elements have `float: right;`, the element that appears FIRST in the HTML gets the RIGHTMOST position.

### GENERAL RULE

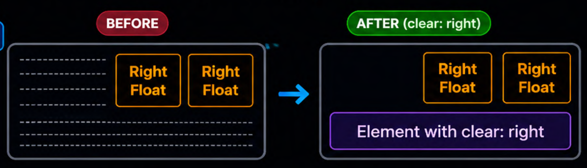
HTML Order : A B C

`float: right;` for all

Visual Order : C B A

# Clear CSS Property

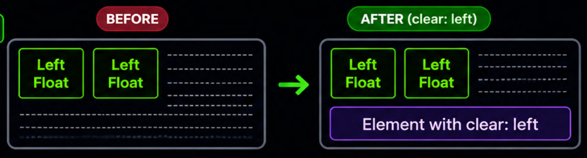
**1** **clear: right**


BEFORE
Right Float
Right Float
AFTER (clear: right)
Right Float
Right Float
Element with clear: right

**clear: right**

Moves element below all right-floating elements.

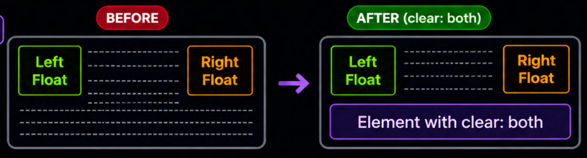
**2** **clear: left**


BEFORE
Left Float
Left Float
AFTER (clear: left)
Left Float
Left Float
Element with clear: left

**clear: left**

Moves element below all left-floating elements.

**3** **clear: both**


BEFORE
Left Float
Right Float
AFTER (clear: both)
Left Float
Right Float
Element with clear: both

**clear: both**

Moves element below all left and right floating elements.

🟩 **LEFT FLOAT** Element floated left

🟧 **RIGHT FLOAT** Element floated right

🟪 **BOTH (LEFT + RIGHT)** Element moved below both sides

### In Simple Words:

**When `float: right` is applied to **Element-1**, the element behaves as if it is saying:**

**"Move me to the right side of my container. Also, if there is any empty space available on my left side, other elements are allowed to flow into and occupy that space. I have no problem sharing that available area with them."**

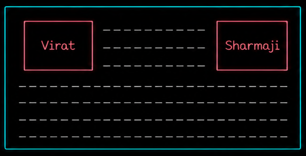
**The same idea applies to `float: left`:**

**"Move me to the left side of my container. If there is any empty space available on my right side, other elements may flow into and use that space."**

**When `clear: right` is applied to **Element-1**, the element behaves as if it is saying:**

**"If there is any element floating on my right side, do not allow me to stay beside it. Move me downward until there are no right-floated elements next to me."**

**Q1)**


Virat
Sharmaji

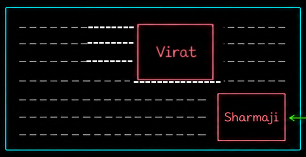
```
<body>
  <img -> sharmaji
  <img -> virat
  <p
</body>
```

Now let Sharmaji image be `float: right`

`Clear: left` applied. What will happen?

```
img1 {
   float: right
}
img2 {
   float: right
}
```

=


Virat
Sharmaji

← Wrong.

★ **Bcoz** `<body>`

`<img src="sharmaji.jpg">`

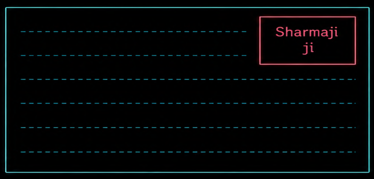
**First, this statement will be processed:**

**"Take Sharmaji's image and float it to the right. If there is any floating element on my left side, move me below it."**

**However, when the browser processed this statement, there was no floating element on Sharmaji's left side.**

**Therefore, Sharmaji's image was not moved downward.**

**So, at the moment this statement was evaluated, the condition looked like this:**

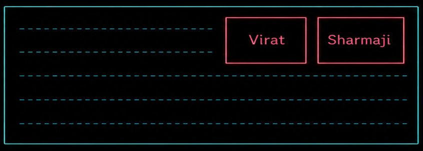

Sharmaji ji

As

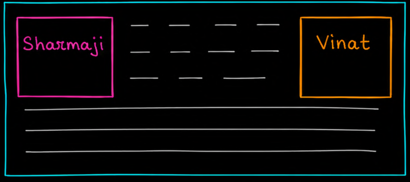
```
<img src="virat.jpg">
<p> ... </p>
```

↓

This state executes.


Virat
Sharmaji

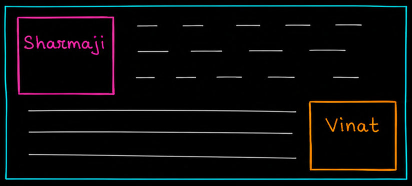
**Q2)**


Sharmaji
Virat

```
img1 {
    float: left;
 }
img2 {
    float: right;
 }
```

**If now virat** has `clear: left` then what will happen.

=


Sharmaji
Virat

✓